Agile Modeling, Prototyping, and Scrum

CSE 4407

Md. Bakhtiar Hasan

Assistant Professor
Department of Computer Science and Engineering
Islamic University of Technology

July 16, 2025

What is Prototyping?

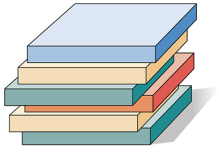
- Creating an early, working or non-working version of an information system (or parts of it)
- Key Goal: Elicit user and management reactions *before* final development
  - How do they *feel* interacting with it?
  - Does it *meet* their needs? $\rightarrow$ Goodness-of-fit
- Methods for Feedback: Observation, Interviews, Feedback Sheets/Questionnaire
- Prototyping and planning go hand-in-hand $\rightarrow$ Allows for inexpensive adjustments

Why Bother Prototyping?

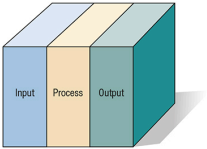
- Gather Realistic Feedback: Users interact with *something tangible*, not just abstract ideas
- Set Priorities Effectively: Feedback highlights what's *really* important to users
- Redirect Plans Inexpensively: Cheaper to change a prototype than a fully coded system $\rightarrow$ Prototype twice, code once?
- Reduce Misunderstandings: A visual model clarifies requirements better than lengthy documents alone
- Minimize Disruption: Early problem detection prevents bigger issues later

Flavors of Prototypes

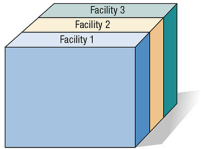
- Prototyping isn't a single technique; different approaches suit different situations
- Four main types
  - Patched-Up
  - Nonoperational
  - First-of-a-Series
  - Selected Features



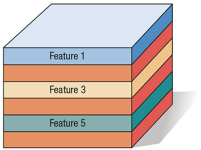
Patched-Up Prototype



Nonoperational Prototype



First-of-a-Series Prototype



Selected Features Prototype

Patched-Up Prototype: It Works... Mostly!

- Concept: A working system, but constructed inefficiently or “patched together”
- Analogy: “Breadboarding” in engineering $\rightarrow$ Making a functional but messy circuit model
- Focus: Allows users to interact with the interface and outputs, get a feel for functionality
- Catch: Performance (speed, storage) is often poor because it was built quickly, not optimally
- Why: Good for testing core logic and user interaction flow when efficiency isn’t the immediate concern

Nonoperational Prototype: All Show, No Go?

- Concept: A non-working scale model focusing on the look and feel (Input/Output)
- Analogy: A full-scale car model for wind tunnel tests $\rightarrow$ Looks like a car, but has no engine
- Focus: Tests design aspects, User Interface (UI), and system navigation without functional backend processing
- Why: When backend coding is too complex, costly, or time-consuming to prototype, but UI/UX feedback is vital
- Users can still evaluate the system's usability and usefulness based on the simulated interaction

First-of-a-Series: The Pioneer System

- Concept: A fully operational, complete, full-scale working model $\rightarrow$ The Pilot
- Analogy: The first airplane built in a new series - tested thoroughly before mass production
- Purpose: Used when the *same system* will be installed in many locations or for many users
- Benefit: Allows realistic user interaction and identifies problems *before* widespread rollout
- Example: Implementing a new inventory system in *one* store of a large retail chain first $\rightarrow$ Work out the kinks there before rolling it out to hundreds of stores

Selected Features: Build and Test in Chunks

- Concept: An *operational* model including *some*, but not all, features of the final system
- Analogy: A new shopping mall opening with only some stores ready, while others are still under construction
- Process: Build and deliver the system in functional modules $\rightarrow$ Users test these modules
- Benefit: Successful, user-approved features can be directly incorporated in the final system $\rightarrow$ Reduces rework
- Example: A user menu shows 6 options, but only `Add Record`, `Delete Record`, and `List Record` are currently working in the prototype
- CORE: This is the most common type referred to when discussing prototyping in agile contexts

The User's Role: The VIP of Prototyping!

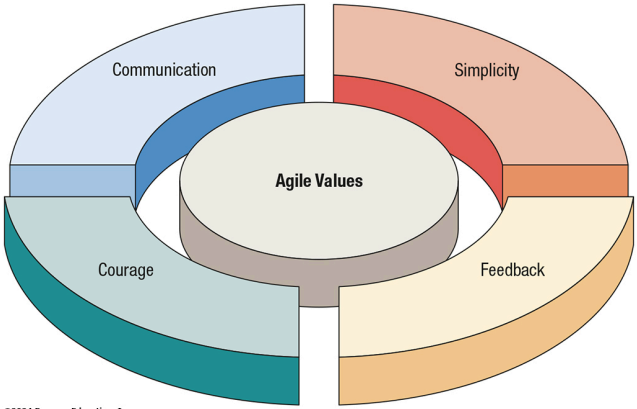
- User involvement isn't optional → Prototyping without users is pointless!
- What's Required? → "Honest Involvement"
  - Interact genuinely with the prototype
  - Provide candid feedback (good and bad)
  - Suggest changes or new ideas (innovations)
- Analyst's Role
  - *Encourage* and welcome feedback → Avoid being defensive!
  - *Observe* user reactions carefully
  - *Translate* feedback into workable system changes
  - *Communicate* the prototype's purpose clearly

Agile Modeling: The Core Idea

- A collection of **innovative**, **user-centered** approaches to systems development
- More than a process: Based on shared **values** and guiding **principles** [1]
- Focuses on collaboration, rapid cycles, and adapting to change
- Often contrasted with traditional “plan-driven” or “waterfall” methods

The Foundation: Four Core Values

- Agile isn't just about *what* you do, but *how* and *why* you do it
- Crucial to create the environment for success



Communication: Talk is *Not* Cheap!

- Why
  - Systems development is complex
  - Misunderstandings are easy $\rightarrow$ Jargon, Deadlines, Distributed teams
- Agile Solution: Prioritize constant, clear communication
  - Practices like *Pair Programming* inherently foster communication
  - Frequent interaction helps fix problems quickly
- Avoids
  - Solving the wrong problem
  - Managers misunderstanding tech issues
  - Knowledge loss when team members change

Simplicity: Keep It Simple! (KIS)

- Goal: Address complexity by doing the *simplest* thing that works right now
- Mindset: Understand that today's simple solution might need tweaking tomorrow $\rightarrow$ and that's OK!
- Requires a laser focus on the immediate project goals
- Avoids "Gold Plating" (adding unnecessary features) and over-engineering
- Analogy: You can't run before you can walk, can't walk before you can stand

Feedback: Constant Course Correction

- Tied closely to **Time** $\rightarrow$ Feedback loops operate on different scales (seconds, days, weeks)
- Forms of Feedback
  - Technical: Automated tests breaking code (fast!)
  - Functional: Customers testing implemented “user stories”
  - Process: Comparing plan progress to actual results
- Purpose: Allows developers and customers to make adjustments *early* and *often*
- Customers experience parts of the system early, ensuring it aligns with their needs

Courage: Trust and Boldness

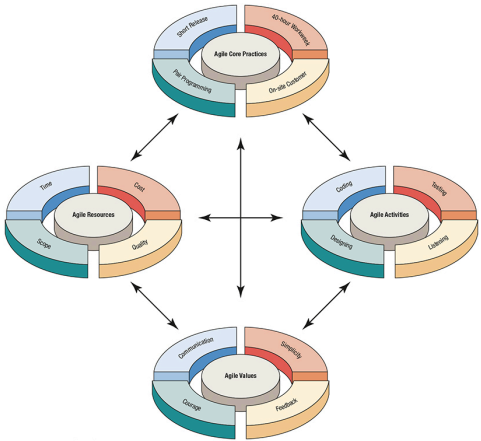
- Foundation: High level of trust and psychological safety within the team and with customers
- Manifestations
  - Willingness to **throw away** code that isn't working and start over
  - Trusting you instincts (and test results!)
  - Acting on feedback, even if it means rethinking a solution
  - Experimenting with simpler/better approaches
- Requires: Trusting teammates and customers enough to make potentially disruptive (but ultimately beneficial) changes
- Underpinned by Humility: Acknowledge you don't have all the answers
  - Be open to critique and learning from domain experts (users/customers)
  - Avoid the "guru" complex!

Agile Principles: The Guiding Rules

- Principles translate the core *values* into more concrete *guidelines*
- Help teams measure if they are truly *acting* in an Agile way
- Differentiate Agile from traditional methodologies
- Often expressed as memorable sayings or proverbs (14+ common ones exist)
  - *Satisfy the customer* through early and continuous delivery of working software
  - *Embrace changing requirements*, even late in development
  - *Deliver* working software *frequently*
  - Business people and developers must *work together daily*
  - Build projects around *motivated individuals* $\rightarrow$ *Trust* them
  - *Face-to-face conversation* is the most effective communication

Putting Agile Together: Key Components

- Agile isn't just values and principles; it involves specific ways of working
- Three Key Aspects
  - Activities: The fundamental things developers do
  - Resources: The levers project managers can adjust
  - Practices: Specific techniques unique to agile



Agile Activities: The Daily Grind

- Coding
  - The one indispensable activity
  - Source of learning [1]
  - Communicates ideas concretely [1]
- TestingTesting
  - Critical!
  - Automated tests are key (unity, functional, etc.)
  - Provides confidence
  - Ensures long-term maintainability
- ListeningListening
  - Paramount skill (esp. with less formal documentation)
  - Active listening to partners and customers (assume you know nothing about their business!)
- DesigningDesigning
  - Creating structure $\rightarrow$ Keep it simple, flexible
  - Evolutionary process $\rightarrow$ Design emerges and improves

Balancing Act: Resource Control Variables

- Projects always have constraints $\rightarrow$ Agile acknowledges these and provides levers to pull
- Four Key Variables
  - Time: The deadline
  - Cost: The budget
  - Quality: How good does it need to be?
  - Scope: What features are included?
- Key Idea: You can rarely fix all four
  - Agile often prioritizes **Time** and adjusts **Scope**
  - Trade-offs are necessary!

Controlling Time and Cost

- Time
  - Agile challenges extending deadlines. Finishing *on time* is highly valued
  - Often, delivering *core functionality* on schedule is better than being late with everything
- Cost
  - Adding more people (Cost) doesn't always reduce Time proportionally
    - Brook's law: Adding manpower to a late software project makes it later.
    - Communication overhead
  - Chronic overtime (Cost) is counter-productive $\rightarrow$ Fatigue leads to errors, reducing quality
  - Wise Cost Investments: Tools (CASE, Visio), better hardware

Controlling Quality and Scope

- Quality
  - Agile distinguishes
    - Internal Quality: Functionality, maintainability, conformance → Generally not compromised
    - External Quality: Customer perception (performance, UI polish, minor bugs)→ May be adjusted to meet deadlines
  - Frequent updates in COTS software often reflect this trade-off
- Scope
  - The primary control lever in many Agile projects
  - Determined by customer “stories” (more on this soon)
  - If time/cost are fixed, Scope is adjusted
    - Features are prioritized
    - Lower-priority ones might be deferred to a later release

Agile Practices: Making It Happen

- Specific techniques that embody the values and principles
- Core Practices of Extreme Programming (XP)
  - Short Releases
    - Deliver working software frequently (weeks, not months)
    - Tackle highest priority features first
  - 40-Hour Workweek
    - Intense, focused work during normal hours
    - Avoid burnout, reduces errors $\rightarrow$ Chronic overtime is a danger sign
    - Sustainable Pace
  - On-site Customer
    - A real business expert embedded with the dev team daily
    - Answers questions, clarifies stories, sets priorities
  - Pair Programming
    - Two developers, one computer
    - Code together, test together
    - Improves quality, shares knowledge, sparks ideas

A Typical Agile Flow

- Agile modeling is iterative and embraces change
- A common flow
  - **Listen** for User Stories $\rightarrow$ Customer needs
  - **Draw/Model** workflow $\rightarrow$ Understand the business logic
  - **Refine/Create** Stories based on model
  - Develop **Prototypes** for feedback (e.g., UI mockups)
  - **Use Feedback** (from models and prototypes) to Code, Test, Design iteratively
  - **Repeat** $\rightarrow$ Deliver working software incrementally
- "**Agile**" = **Maneuverability** $\rightarrow$ Ability to react quickly to change and feedback

Example: Online Media Store User Stories (1/2)

- Short, User-focused, and Value-oriented ones:
  - Welcome the customer $\rightarrow$ If returning, welcome them back
  - Show specials $\rightarrow$ Show new products, tailor if recognized
  - Search $\rightarrow$ Find specific product and similar ones
  - Show results $\rightarrow$ Display search matches clearly
  - Allow detail view $\rightarrow$ Offer sample pages, video clips, etc.
  - Display reviews $\rightarrow$ Show comments from other customers
  - Place in cart $\rightarrow$ Easy button to add item to cart
- Enough to understand the need, but not enough detail to code directly (yet!)

Example: Online Media Store User Stories (1/2)

- Continuing the purchase process
  - Keep purchase history → Remember customer details/purchases
  - Suggest similar → Show related items
  - Proceed to checkout → Confirm customer identity
  - Review purchases → Allow final check before paying
  - Continue shopping → Option to add more items
  - Shortcut checkout → Faster process for known customers/addresses
  - Gift options → Add recipient details, gift card message
  - Shipping options → Choose method based on cost/speed
  - Complete transaction → Finalize payment
- Covers the flow, but each requires further discussion before implementation [2]

Managing the Stories: Tools

- While simple index cards work, software tools are common for managing stories, especially in larger projects
- Examples: Jira, ADO Planbox, ScrumDesk, Digital.ai, etc. $\rightarrow$ Market changes frequently!
- What to look for
  - Easy setup, learning, and use
  - Customizable
  - Features: Story/Epic management, estimation (story points), decomposition, linking, etc.
- Preference for *understandable* tools over overly complex ones

Anatomy of a Story: Card View

Need or Opportunity	Apply shortcut methods for faster checkout				
Story	If the identity of the customer is known and the delivery address matches, speed up the transaction by accepting the credit card on file and the rest of the customer's preferences such as shipping method.				
	Well Below	**Below Average**	**Average**	**Above Average**	**Well Above**
Activities	**Coding**		✓		
	Testing		✓		
	Listening		✓		
	Designing			✓	
Resources	**Time**			✓	

- Captures the core need and acts as a token for conversation
- May include initial rough estimates or notes about complexity/resources

Important Reminder: Stories != Specs

- User stories are intentionally **brief**
- Their **low level of detail** is a common source of difficulty if not addressed $\rightarrow$ Research insight
- They **reduce** initial focus on **detailed specification**, which MUST be compensated for thorough
  - Ongoing conversation with the customer/user
  - Collaboration within the development team
  - Defining clear **Acceptance Criteria** (how we know the story is “done”)
- User stories are the **start** of requirements discovery, not the end!

Scrum: The Core Idea

- An Agile framework for managing complex projects
- Emphasizes teamwork, self-organization, and accountability
- Work progresses in iterative cycles called **Sprint** $\rightarrow$ Typically 2-4 weeks
- Each Sprint aims to produce a potentially shippable product increment
- Analogy
  - Rugby team has an overall strategy, but the team adapts tactics *during the game* based on the situation
  - Project Owner sets vision, team figures out *how* to deliver

The Scrum Team: Key Roles

- Three specific roles
  - Product Owner: Represents the customer and business value
  - Scrum Master: Facilitates the process and removes impediments
  - Development Team: Builds the product (cross-functional)
- These roles collaborate closely throughout the Sprint

Product Owner - The Visionary

- Owns the “What” and “Why”: Defines the product vision and features
- Analogy: The “Project Creator” or the ship’s captain setting the destination
- Manages the Product Backlog: Creates, prioritizes, and clarifies user stories
- Represents the Business/Customer: Makes decisions on priorities to maximize value
- Responsible for releases and final assessment of the product

Scrum Master - The Facilitator and Coach

- Owns the “How” (process): Ensures Scrum practices are understood and followed
- Servant Leader: Coaches the team, facilitates meetings, helps them improve
- Impediment Remover: Protects the team by removing obstacles (technical, organizational, etc.) $\rightarrow$ The “jungle guide clearing the path”
- Fosters the team culture, moderates conflict
- NOT a traditional Project Manager (doesn’t assign tasks)

Development Team - The Builders

- Owns the “How” (work): Figures out how to turn Product Backlog items into a working product increment
- Cross-functional: Possesses all skills needed
- Self-organizing: Decides how best to accomplish the work; collaborates intensely
- Responsibility
  - Refining stories
  - Estimating effort
  - Completing the work during the Sprint
- Typically 3-9 members

The Product Backlog

- An **ordered list** of everything known to be needed in the product
- Analogy: A restaurant's entire menu
- Source of all requirements (features, functions, fixes, enhancements, etc.) $\rightarrow$ Typically expressed as User Stories
- Dynamic: Constantly evolving as needs change and understanding growth
- Prioritized by the Product Owner: Most important items are at the top
- Items need to be clear enough ("Ready") before being pulled into a Sprint

Product Backlog Registry Example

- Shows how stories are tracked with key information
- Includes acceptance criteria $\rightarrow$ Vital for knowing when a story is “Done”

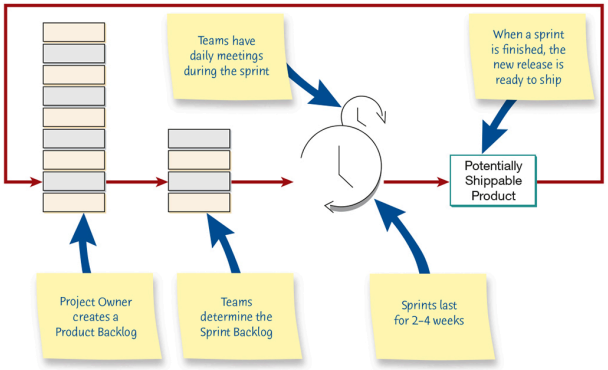
Number	Tasks based on user stories	Who will benefit	Resources required	Will be accepted when
W212	Make it easier to continue shopping once an item is placed in the shopping cart	Visitors to the website	Minor coding effort, testing	Users react positively to the prototype, coding is completed and tested
W210	Offer options for shipping earlier in the online ordering process	Visitors to the website	Dynamically calculating rates based on zip code and preferred delivery speed, coding, testing	Validity is checked, code is tested, information is approved as understandable to users
W211	Add product reviews by peers	Users who would feel more confident having these reviews available	Collaborative filtering, redesign of product pages, coding, testing	Users approve of the prototype and the collaborative system is thoroughly tested

The Sprint Cycle

- Sprint Backlog: The subset of Product Backlog items selected by the *team* for completion in one Sprint
- Analogy: If the Product Backlog is the menu, the Sprint Backlog is the specific order placed by the team for the next two weeks
- Sprint Length: Fixed timebox (e.g., 2 weeks) $\rightarrow$ Consistent length helps establish rhythm
- Sprint Goal: A clear objective for what the Sprint aims to achieve $\rightarrow$ Provides focus
- Output: A “Done”, usable, potentially shippable product increment
- End-of-Sprint Check
  - Does the increment add value?
  - Is it genuinely ready (tested, documented, integrated)?

The Scrum Flow

- Shows the flow from backlog creation to shippable product within repeating cycles
- Emphasizes the roles and key events within the cycle



Scrum Events: Keeping the Rhythm

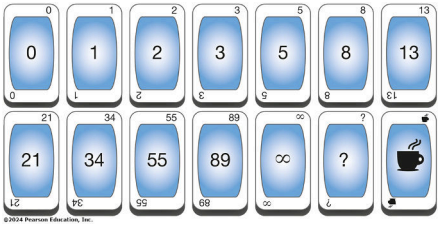
- Scrum uses specific timeboxed events (meetings) to create regularity and minimize other meetings
- Key Events within a Sprint
  - Sprint Planning: What can be done this Sprint? How?
  - Daily Scrum: What did I do? What will I do? Any blockers? (15 min sync)
  - Sprint Review: What did we accomplish? Demo and feedback
  - Sprint Retrospective: How can we improve the process?

Event: Sprint Planning

- Purpose: Plan the work to be performed in the Sprint
- Part 1 (What)
  - Product Owner presents prioritized backlog items
  - Team asks clarifying questions
  - Sprint Goal is crafted
- Part 2 (How)
  - Team selects items they forecast they can complete
  - Decompose stories into smaller tasks
  - Team commits to the Sprint Goal
- Estimation: Techniques like Planning Poker are often used here to estimate effort

Planning Poker: Fun with Estimation!

- A consensus-based estimation technique
- Uses cards with numbers (often Fibonacci sequence) representing relative effort (Story Points) or time
- Process
  - PO explains a story
  - Team discusses briefly
  - Everyone privately chooses a card reflecting their estimate
  - Cards revealed simultaneously (avoids anchoring)
  - Discuss differences (especially highest/lowest estimates)
  - Repeat voting until consensus is reached
- Makes estimation collaborative and often more accurate

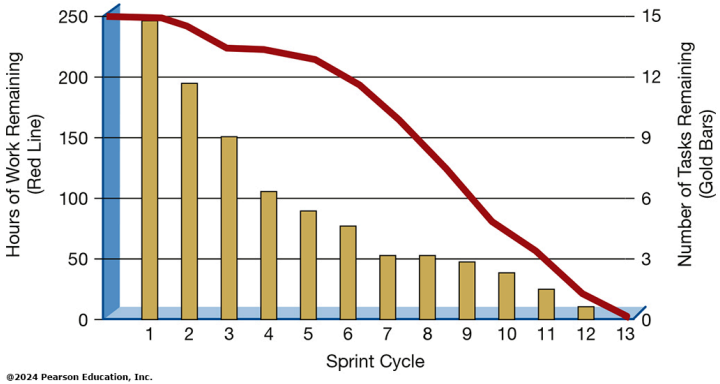


Event: Daily Scrum/Stand-up

- Purpose
  - Quick synchronization for the Development Team
  - Inspect progress toward Sprint Goal and adapt plan for the next 24 hours
- Format
  - Same time, same place each day
  - Max 15 minutes $\rightarrow$ Often standing up to keep it brief!
- Each team member answers 3 questions:
  - What did I do yesterday to help meet the Sprint Goal?
  - What will I do today to help meet the Sprint Goal?
  - Do I see any impediments blocking me or the team?
- NOT a status report for managers $\rightarrow$ Planning meeting for the Team
- Impediments are identified, resolved *after* the Daily Scrum

Visualizing Progress: Burndown Chart

- Showing work remaining in the Sprint Backlog over time
- Helps visualize if the team is on track to complete the forecasted work by the end of the Sprint $\rightarrow$ Updated daily



End of Sprint: Inspect and Adapt

- Two distinct meetings are held, both at the end of a sprint
- Sprint Review
  - Purpose
    - Show stakeholders **what** was accomplished
    - Get feedback
  - Activities
    - Team demonstrates the “Done” work
    - Product Owner discusses backlog
    - Collaborative feedback session
- Sprint Retrospective
  - Purpose
    - Reflect on **how** the Sprint went
    - Identify process improvements
  - Activities
    - Team discusses what went well, what didn’t, and what to change for the *next* Sprint
    - Focus on the team and process

Side Note: Kanban - Visualizing Workflow

- Concept from Toyota manufacturing, now widely used in software → Means “Signboard”
- Key Principles
  - Visualizing the workflow (e.g., on a board)
  - Limit Work In Progress (WIP) → Don’t start too many things at once!
  - Manage flow (identify and remove bottlenecks)
  - Make process policies explicit
  - Implement feedback loops
  - Improve collaboratively, evolve experimentally
- Helps make bottlenecks and process issues visible
- Note: Kanban can be used *with* Scrum or as a standalone method

The Kanban Board

- Cards (physical or digital) represent work items (stories, tasks)
- Items are “Pulled” from one column to the next when capacity exists

Kanban Metrics and Scrumban

- Kanban helps track flow metrics
  - Cycle Time: Average time an item takes from start to ‘Done’
  - Throughput: Average number of items completed per time period $\rightarrow TP = WIP/CT$
- Scrumban: Using Kanban principles *within* the Scrum framework
  - Visualizing the Sprint Backlog on a Kanban board
  - Using WIP limits during the Sprint
  - Focusing on flow within the fixed Sprint timebox

Why Use Scrum? The Advantages

- Speed: Quick cycles deliver value faster
- User Focus: On-site customer and reviews keep it aligned
- Teamwork: Strong emphasis on collaboration and shared ownership
- Clarity: Simpler roles/process than some traditional methods
- Flexibility/Adaptability: Embraces change via short cycles and feedback
- Motivation: Frequent small wins, team empowerment
- Feedback: Built-in loops (Daily Scrum, Review, Retro)

Scrum Challenges and Considerations

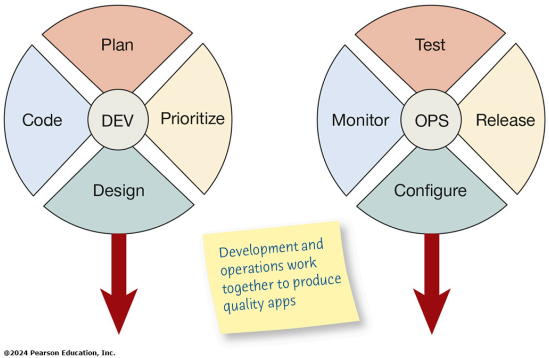
- Documentation: Can be neglected if not explicitly planned for
- Quality: Risk of errors if “Done” isn’t rigorous or pressure leads to shortcuts
- Scope Creep: Requires strong Product Owner to manage the backlog effectively
- Pressure
  - Short sprints can feel intense
  - Risk of burnout if pace isn’t sustainable
- Team Challenges
  - Difficult for geographically dispersed teams
  - Requires good communication skills
  - Hard to integrate highly specialized skills if needed part-time
  - Replacing members mid-stream is disruptive
- Organizational Support: Requires organizational buy-in and discipline to work well

DevOps: Development + Operations

- Analogy: Having architects work closely with the construction crew and maintenance staff, instead of just tossing blueprints over the wall.
- Goal: Decrease deployment time for new applications
- Benefit: Respond rapidly to market opportunities and customer feedback $\rightarrow$ Maximize value/profit
- Approach: Tries to merge Development and Operations processes and mindsets
- Focus on automation, collaboration, and shared responsibility

More Than Tools: It's a Culture

- DevOps is primarily a **cultural shift**, requiring a different **mindset**
- Avoids the "blame game" between Dev and Ops when issues arise $\rightarrow$ Focus on fixing the problem together
- Often creates parallel streams... but working **together** towards shared goals
- Collaboration and shared responsibility are key



Organizing for DevOps Flow

- Work organized around **Value Streams** $\rightarrow$ The sequence of activities to deliver value to the customer
- Small, empowered teams (5-10 people) responsible for a specific value stream or service
- The “Two-Pizza Rule” (Amazon)
  - If you can’t feed the team with two large pizzas, the team is probably too big for effective collaboration
  - Keeps communication overhead low

How DevOps Teams Work (1/2)

- Limit Multitasking: Focus on 1-3 projects/initiatives at most → Avoids context-switching penalties
- Small Batches, Small Steps
  - Deliver work in small, manageable increments
  - Easier to test, deploy, and roll back if needed
- Reduce Handoffs
  - Minimize dependencies on external teams or departments
  - Empower the team to do more end-to-end
- Eliminate Waste: Actively identify and remove
  - Partially completed work → Doesn't deliver value
  - Extra, unnecessary processes or features
  - Connection: Relates to Lean principles often associated with Kanban
  - Minimize Task Switching
  - Reduce effort spent Tracking Down Defects

How DevOps Teams Work (2/2)

- “Swarm” Problems
  - When a problem occurs, the team collaborates immediately to fix it and understand the root cause
  - Prevents issues from escalating
  - Fosters learning
- Quality is Built-In
  - Responsibility for quality rests with the team doing the work, not a separate QA department gatekeeping at the end
  - Automated testing is key

DevOps: Embracing Failure to Improve

- Analogy: Like practicing emergency drills (fire, earthquake) so you know how to react when a real one happens.
- Focus on building **resilient systems** that can handle failures gracefully
- Proactively test resilience through “Fire Drills” or “Chaos Engineering”
  - Example: Netflix’s “Chaos Monkey” randomly disables production servers to ensure the system can cope
- Integrate **Security** practices early and continuously $\rightarrow$ DevSecOps
- Learn from failures
  - Debrief after drills (success/failure? met goals?)
  - Conduct blameless **postmortems** after real incidents/breaches to understand root causes and improve

Agile: Addressing Traditional Concerns

- Agile Recap
  - Focus on rapid, iterative delivery
  - Direct user involvement
  - Flexibility
  - Quality through feedback and collaboration
  - Tweaking is expected
- Common SDLC Criticisms Agile Addresses
  - Often perceived as too **time-consuming** and bureaucratic
  - Can be too focused on **data**/**process** over human needs and usability
  - Can be **costly** due to lengthy cycles and difficulty adapting to change late in the project
- Agile aims to be **responsive** to changing requirements, business conditions, and user needs

Key Lessons from Agile Practice

- Short Releases Allow Systems to Evolve: Frequent updates enable rapid change incorporation and organic growth based on real user needs
- Pair Programming Enhances Quality: Promotes communication, shared understanding, customer focus, continuous testing, and integration
- On-site Customers are Mutually Beneficial: Provides instant clarification, reality checks, and builds empathy between developers and users
- 40-Hour Workweek Improves Effectiveness: Sustainable pace prevents burnout, reduces errors, and maintains long-term productivity and quality
- Balanced Resources and Activities Support Goals: Consciously managing trade-offs between Time, Cost, Quality, Scope, and Coding, Testing, Listening, Designing is vital
- Agile Values are CRUCIAL: Communication, Simplicity, Feedback, and Courage are not just buzzwords; commitment to them is essential for success

Framework for Comparison: Efficiency

- Analyzes impact of Agile vs Structured Methods on efficiency of “knowledge work”
- Davis and Naumann (1999) identified 7 strategies for improving knowledge work efficiency
- Identified huge productivity variance in programming vs. other work $\rightarrow$ Best devs 5-10x more productive than worst
- Suggested methodology/tools matter greatly

Comparison: Interfaces, Learning, and Errors

- Reduce Interface Time and Errors
  - Structured
    - Relies on **Standards** (coding, naming), common **Forms**, defined procedures
    - Aims for consistency upfront
  - Agile
    - Uses **Pair Programming** for real-time checking and shared understanding
    - Aims for quality through collaboration
- Reduce Process Learning and Dual Processing
  - Structured: Manages releases carefully to minimize users learning multiple systems simultaneously
  - Agile
    - Focuses on **Rapid Development** and prototyping
    - Often less emphasis on complex tool learning (e.g., CASE) upfront
    - Faster feedback reduces dual processing needs

Comparison: Structuring Tasks and Parkinson's Law

- Reduce Task Structuring and Formatting Time
  - Structured
    - Uses **CASE tools**, **diagrams** (DFDs, ERDs), templates, code reuse to structure work upfront
  - Agile
    - Structures work via **Short Releases**
    - The iteration itself defines manageable chunks
- Reduce Nonproductive Work Expansion $\rightarrow$ Parkinson's Law
  - Structured
    - Relies on **Project Management** techniques and setting (often distant) deadline
  - Agile
    - Uses **Short Releases** with limited scope
    - Deadlines are always imminent, forcing focus and realism

Comparison: Finding Info and Talking to People

- Reduce Data/Knowledge Search Time and Cost
  - Structured
    - Uses **Structured data gathering** techniques → Formal interviews, observation, methods like STROBE, sampling
    - Aims for systematic coverage
  - Agile
    - Relies heavily on the **On-site Customer** for immediate access to information and context
    - Potential risk if customer info is flawed
- Reduce Communication and Coordination Time and Cost
  - Structured
    - Breaks projects into smaller tasks/teams
    - Sometimes uses barriers (e.g., limiting programmer-user contact) → Potential effectiveness cost
  - Agile
    - Uses **Timeboxing** (Sprints, Daily Scrums) to focus communication
    - Values direct team communication, potentially increasing interaction time but aiming for higher effectiveness

Comparison: Handling Information Overload

- Reduce Losses from Human Info Overload
  - Structured
    - May **Filter information** reaching developers (e.g., shield from direct user complaints)
    - Aims to protect focus
  - Agile
    - Emphasizes **Sustainable Pace** (40-hour week)
    - Assumes rested developers handle information better and product higher quality work
    - Avoids overload-induced errors
- Overarching View: Agile is fundamentally **human-oriented**, aiming for solutions through collaboration and adaptation, not just formal specifications

Adopting Agile: Considering the Risks

- Adopting any new methodology, including Agile, involves organizational change and inherent risks
- Risk to consider
  - Organizational Culture
  - Timing
  - Cost
  - Clients' Reactions
  - Measuring Impact
  - Individual Rights
- Need careful consideration before widespread adoption

Risks: Organizational Factors

- Organizational Culture
  - Is the overall culture conservative or innovative? → Agile thrives better in innovative, adaptable cultures
  - Poor fit can lead to resistance and failure
  - Hybrid approaches (mixing Agile and SDLC) might be suitable for some orgs
- Timing
  - Is *now* the right time? → Consider other ongoing projects, strategic initiatives, market conditions
  - Don't introduce major change during peak instability
- Cost
  - Training developers, Scrum Masters, Product Owners
  - Potential consultant fees
  - Opportunity cost → Time spent learning vs. Doing project work

Risks: External and Impact Considerations

- Clients' Reactions
  - Will clients (internal or external) embrace the higher level of involvement required (e.g., On-site Customer, frequent reviews)?
  - Some may resist being part of an "experiment" with new methods $\rightarrow$ Requires clear communication and strong relationships
- Measuring Impact
  - How will success be measured?
  - Is there empirical proof it's better for this context?
  - Agile's history is shorter than SDLC's $\rightarrow$ Less long-term data
  - Risk of integration issues with legacy systems or processes
- Individual Rights/Preferences
  - Agile practices like Pair Programming might clash with preferences of developers who work best alone
  - Need to balance methodology adherence with respecting individual creativity and working styles

References I

[1] K. Beck and C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd. Boston, MA: Addison-Wesley Professional, 2004 (cit. on pp. 10, 18).

[2] H. F. Soares, N. S. Alves, T. S. Mendes, M. Mendonça, and R. O. Spínola, "Investigating the link between user stories and documentation debt on software projects," in *2015 12th International Conference on Information Technology - New Generations*, IEEE, 2015, pp. 385–390. DOI: 10.1109/ITNG.2015.68. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7113503 (cit. on p. 25).